Подробный разбор: архитектура и оркестратор

1. Что мы строим

LLM сама по себе — это одна функция. Она принимает список сообщений (системная инструкция, реплики пользователя, свои прошлые ответы) и возвращает один новый ответ. Между вызовами она ничего не помнит, сама не читает файлы, не запускает команды и не знает, что было в других чатах. Всё, что модель «знает» о задаче, нужно каждый раз заново передать ей во входном списке сообщений.

Harness — это программа вокруг этой функции. Она принимает сообщение пользователя, собирает для модели нужные сведения, вызывает модель, выполняет действия, которые модель просит сделать (прочитать файл, найти строку), сохраняет всё в базу и возвращает пользователю результат. Модель мы не обучаем и не меняем — берём готовую и обращаемся к ней по API.

Особенность нашего harness — уточняющие вопросы. Если пользователь написал «Проверь тесты проекта», а к чату привязано два проекта, обычная модель часто сама выберет один и начнёт работать с ним. Наша цель — чтобы такого не было. Система сначала пытается найти ответ в известных данных (в переписке, в проекте). Если не выходит — задаёт пользователю вопрос «Какой проект проверить?», ждёт ответа и только потом выполняет задачу. Неважные вещи можно принять по умолчанию, но тогда это допущение нужно явно написать в ответе.

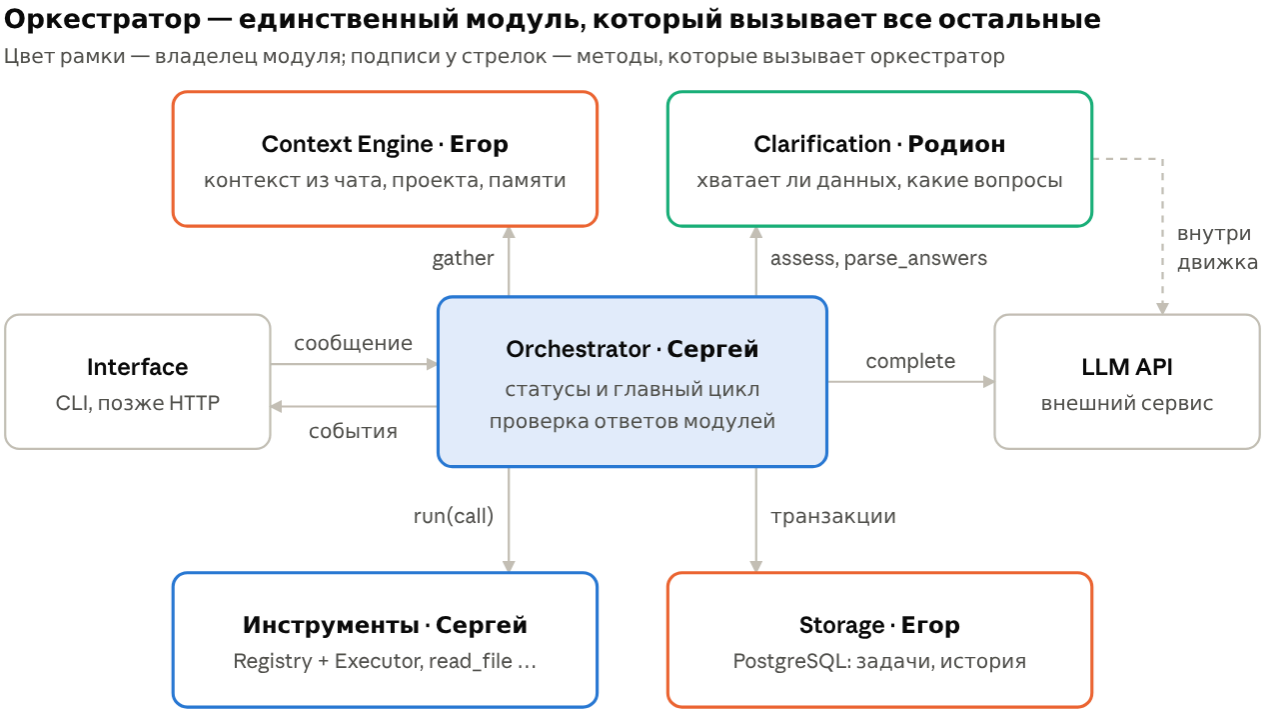
Из этого следует главное требование к архитектуре. Обработка одной задачи может прерваться на неопределённое время: пользователь ответит через минуту или через день. Значит, состояние задачи нельзя держать в оперативной памяти процесса. Оно целиком хранится в базе данных, и любая задача может быть продолжена с того места, где остановилась.

В документе везде используется один пример: пользователь пишет «Проверь тесты проекта», к чату привязаны проекты `backend` и `frontend`.

2. Архитектура

Система состоит из шести модулей и внешней LLM. В центре стоит оркестратор: он единственный, кто вызывает остальные модули. Остальные модули друг друга не вызывают и не знают друг о друге. Исключение одно:

Clarification Engine может сам обращаться к LLM, чтобы проанализировать запрос.



компоненты harness и кто кого вызывает

Все стрелки, кроме пунктирной, выходят из оркестратора: модули не знают друг о друге.

Interface (интерфейс)

То, через что пользователь пишет сообщения и читает ответы. В MVP это может быть консольная программа, позже — HTTP-сервер. Интерфейс ничего не решает. Он передаёт сообщение оркестратору вызовом `submit_message` и показывает пользователю события, которые оркестратор вернул: вопросы, ход работы, финальный ответ или ошибку. Владелец не назначен; скорее всего, это ты, потому что интерфейс работает только с оркестратором.

Orchestrator (оркестратор) — твоя часть

Управляет обработкой задачи от начала до конца. Знает, в каком состоянии находится каждая задача, и решает, что делать дальше: собрать контекст, проверить достаточность данных, задать вопрос, вызвать LLM, выполнить инструмент или завершить задачу. Сам он не ищет контекст, не придумывает вопросы и не хранит данные — для этого он вызывает соответствующие модули. Подробно — разделы 4–10.

Context Engine (сбор контекста) — Егор

По тексту задачи находит полезные сведения: сообщения из текущего чата, файлы и заметки проекта, записи долговременной памяти. Отбирает самое релевантное так, чтобы уложиться в заданный размер (бюджет токенов). Отдаёт список «блоков контекста»; у каждого блока есть текст, источник и оценка полезности. Один метод: `gather(query) → ContextBundle`.

Clarification Engine (уточнение) — Родион

Единственный специальный модуль проекта. Получает задачу, контекст и историю и отвечает на вопрос «хватает ли данных, чтобы выполнить задачу?». Возвращает одно из трёх решений: спросить (и какие 1–3 вопроса), выполнять (и какие допущения принять) или отказаться. Вторая задача модуля — разобрать ответ пользователя на вопрос. Два метода: `assess` и `parse_answers`. Модуль ничего не пишет в базу и сам не показывает вопросы пользователю — это делает оркестратор.

Tool Registry + Executor (инструменты)

Registry — список доступных инструментов. У каждого инструмента есть имя (`read_file`), описание для модели и схема аргументов (например, обязательный строковый `path`). Executor принимает запрос на вызов, проверяет аргументы по схеме, выполняет инструмент с ограничением по времени и всегда возвращает результат — либо успешный, либо текст ошибки. Инструменты MVP: `list_files`, `read_file`, `search_files`, `get_project_context`, `search_memory`. Владелец не назначен, но модуль тесно связан с циклом выполнения, поэтому логично делать его тебе.

Storage / Memory (хранение) — Егор

База данных PostgreSQL и код для работы с ней. Хранит задачи и их статусы, все сообщения, вопросы и ответы, историю вызовов инструментов, проекты и их привязку к чатам. Даёт оркестратору методы вида «создать задачу», «добавить сообщение», «прочитать все сообщения задачи» и возможность выполнить несколько таких операций атомарно, в одной транзакции.

LLM (внешний сервис)

Не наш модуль, а чужой сервис, к которому мы обращаемся по сети. С нашей стороны нужен только тонкий адаптер с одним методом `complete(messages, tools) → LLMResponse`. Он превращает наши сообщения в запрос к API

конкретного провайдера, а его ответ — обратно в наш формат. Если провайдера сменим, поменяется только адаптер.

Общие контракты

Чтобы три человека могли писать модули параллельно, заранее фиксируются две вещи:

- **модели данных** — классы `Task`, `Message`, `Question` и т.д., которыми модули обмениваются;
- **интерфейсы модулей** — список методов с типами аргументов и результата, без реализации.

Оркестратор пишется против интерфейсов, а не против конкретного кода Егора или Родиона. Пока их модули не готовы, ты подставляешь простые заглушки с теми же методами (например, хранение в словаре в памяти вместо PostgreSQL). Когда настоящие модули появятся, заглушки заменяются без изменения кода оркестратора.

3. Путь одного запроса шаг за шагом

Пользователь в чате `chat1` пишет «Проверь тесты проекта». К чату привязаны проекты `backend` и `frontend`. Ниже — всё, что происходит, в порядке выполнения.

Ход 1: пришёл запрос

1. **Интерфейс** вызывает `orchestrator.submit_message(chat_id="chat1", user_id="u1", text="Проверь тесты проекта", client_message_id="m-1")`. `client_message_id` — уникальный номер сообщения, который придумывает интерфейс.
2. **Оркестратор проверяет, не дубль ли это.** Ищет `m-1` в таблице `inbox`. Если там уже есть запись, значит интерфейс прислал то же сообщение повторно (например, при обрыве связи). Тогда оркестратор просто возвращает существующую задачу и ничего не делает. Сейчас записи нет.
3. **Проверяет, нет ли в чате незаконченной задачи.** Нет — значит, это новая задача. (Если бы в чате была задача, ждущая ответа на вопрос, сообщение считалось бы ответом — см. ход 2.)
4. **Создаёт задачу.** Одной транзакцией записывает в базу: запись `tasks` со статусом `new`, сообщение пользователя в `messages` (тип `request`), связку `m-1 → task_id` в `inbox` и первый снимок состояния задачи.

5. **Переводит задачу в `analyzing`** и сохраняет новый статус.
6. **Собирает контекст:** вызывает `context_engine.gather(...)` с текстом запроса. Context Engine возвращает, например, предыдущие сообщения чата и заметки. Проект неизвестен, поэтому файлов проекта среди них нет.
7. **Загружает историю задачи** из базы: её сообщения, уже заданные вопросы и ответы (пока их нет), список проектов чата: `[backend, frontend]`.
8. **Спрашивает Clarification Engine:** `clarifier.assess(...)` — передаёт задачу, контекст, историю, список проектов, номер раунда и сколько ещё раз можно спрашивать. Ответ: `ASK`, один вопрос «Какой проект проверить?» с вариантами `backend`, `frontend`. Пробел «не известен проект» помечен как блокирующий.
9. **Проверяет ответ Clarification Engine:** вопросов не больше трёх, нет вопросов про уже известное и т.д. (подробно — раздел 7). Всё корректно.
10. **Сохраняет вопрос и меняет статус.** Одной транзакцией: вопрос — в `clarification_questions` со статусом `pending`, текст вопроса — в `messages`, статус задачи — `waiting_user`, счётчик раундов уточнения — 1.
11. **Возвращает интерфейсу** задачу и событие `questions_asked` с текстом вопроса и вариантами. Интерфейс показывает вопрос. На этом работа оркестратора по этому сообщению заканчивается.

LLM в этом ходе не вызывалась для выполнения задачи. Пока не известно, какой проект проверять, выполнять нечего. В базе задача лежит в статусе `waiting_user` и может ждать сколько угодно.

Ход 2: пользователь отвечает «backend»

1. **Интерфейс** снова вызывает `submit_message(chat_id="chat1", text="backend", client_message_id="m-2")`.
2. **Оркестратор** проверяет дубль (нет) и находит в чате незаконченную задачу в статусе `waiting_user`. Значит, сообщение — это ответ на вопрос.
3. **Разбирает ответ:** берёт из базы открытые вопросы и вызывает `clarifier.parse_answers(...)`. Результат: тип сообщения `ANSWER`, ответ на вопрос про проект = `p_backend`. Другие возможные типы: «отмена», «новый запрос», «непонятно».
4. **Сохраняет ответ.** Одной транзакцией: сообщение в `messages` (тип `answer`), ответ в `clarification_answers`, статус вопроса → `answered`. В задачу записывается параметр `project_id = p_backend` с пометкой «сказал пользователь», статус задачи → `analyzing`.

5. **Снова собирает контекст.** Теперь проект известен, и Context Engine добавляет, например, README проекта `backend`.
6. **Снова спрашивает Clarification Engine** (теперь с пометкой «после ответа»). Ответ: `PROCEED` и одно допущение — «не сказано, какие именно тесты, проверяю все».
7. **Переводит задачу в `executing`**, сохранив допущение в задаче.
8. **Первый шаг выполнения.** Собирает промпт: системная инструкция + описание задачи с известными параметрами + контекст + переписка (вопрос и ответ). Вызывает `llm.complete(промпт, список инструментов)`. Модель отвечает не текстом, а запросом: «вызови `read_file` с аргументом `path="README.md"`».
9. **Выполняет запрошенный инструмент.** Сначала записывает в базу запрос модели и пометку «вызов начат». Затем Executor читает `README.md` в каталоге проекта `backend`. После этого в базу записываются результат, пометка «вызов успешен» и обновлённые счётчики задачи. Интерфейсу уходит событие `progress`.
10. **Второй шаг выполнения.** Снова собирает промпт — теперь в нём есть запрос модели и содержимое README — и вызывает LLM. Модель отвечает обычным текстом без запросов инструментов — значит, работа закончена.
11. **Завершает задачу.** К тексту модели дописывает блок «Допущения: проверяю все тесты». Сохраняет финальный ответ, статус → `completed`, и возвращает интерфейсу событие `final_answer`.

Итог по статусам: `new → analyzing → waiting_user` (конец хода 1) `→ analyzing → executing → completed` (конец хода 2). Полная диаграмма вызовов этого сценария — в начале вкладки «Оркестрация».

4. Оркестратор: что он делает и чего не делает

За что отвечает оркестратор

1. **Принимает сообщения от интерфейса и решает, что это:** новая задача, ответ на вопрос или повтор уже полученного сообщения.
2. **Ведёт задачу по статусам** и следит, чтобы переходы были только разрешёнными (раздел 5).
3. **Вызывает другие модули в нужном порядке:** Context Engine → Clarification Engine → LLM → инструменты, и передаёт каждому ровно те данные, которые ему нужны.

4. **Проверяет ответы модулей** и не даёт ошибке в одном модуле нарушить главное правило: не додумывать критичные параметры.
5. **Собирает промпт для LLM** на этапе выполнения из задачи, контекста и переписки.
6. **Крутит цикл выполнения:** LLM просит инструмент → оркестратор выполняет → отдаёт результат LLM, пока модель не даст финальный ответ (раздел 8).
7. **Сохраняет всё в базу** через Storage после каждого шага, чтобы задачу можно было продолжить после падения процесса (раздел 9).
8. **Следит за лимитами** и обрабатывает ошибки (раздел 10).
9. **Возвращает интерфейсу события:** вопросы, ход работы, итог или ошибку.

Чего оркестратор не делает

- **Не ищет контекст** — это делает Context Engine. Оркестратор только формирует запрос (какой текст, какие источники, какой размер).
- **Не решает, что спрашивать**, и не формулирует вопросы — это делает Clarification Engine. Оркестратор только исполняет его решение и проверяет, что оно не нарушает правил.
- **Не пишет SQL** — все обращения к базе идут через методы Storage.
- **Не держит состояние задачи в памяти** между вызовами. Любой вызов начинается с чтения задачи из базы. Единственное исключение — кеш собранного контекста, чтобы не собирать его заново на каждом шаге выполнения. Его потеря не страшна: контекст просто соберётся снова.

Публичные методы

Это всё, что видит интерфейс:

Метод	Что делает	Что возвращает
`submit_message(msg)`	принимает сообщение пользователя и крутит обработку, пока задача не дойдёт до ожидания ответа или до завершения	задачу + список событий
`cancel(task_id)`	отменяет задачу, даже если она сейчас выполняется	задачу + событие `task_cancelled`

Метод	Что делает	Что возвращает
`resume(task_id)`	продолжает задачу, которая застряла из-за падения процесса	задачу + события
`get_task(task_id)`	читает текущее состояние задачи	задачу

Входное сообщение `msg` содержит: `chat_id` (чат), `user_id` (пользователь), `text` (текст), необязательный `project_id` (если интерфейс уже знает проект) и необязательный `client_message_id` (для защиты от дублей).

Виды событий:

Событие	Когда	Данные
`questions_asked`	задача ждёт ответа	номер раунда, список вопросов (id, текст, варианты)
`progress`	выполнен вызов инструмента	имя инструмента, успех или ошибка, номер шага
`final_answer`	задача выполнена	текст ответа, список допущений
`task_failed`	задачу не удалось выполнить	код причины, описание
`task_cancelled`	задача отменена	причина: пользователь или новый запрос

Внутреннее устройство

Оркестратор состоит из нескольких частей. Каждая разобрана в своём разделе:

Часть	Файл в каркасе	Задача	Раздел
Маршрутизация входящего сообщения	`orchestrator.py`: `submit_message`, `_start_new`, `_handle_answer`	решить, что за сообщение, и создать задачу или сохранить ответ	6
Конечный автомат	`fsm.py`	список статусов и разрешённых переходов	5

Часть	Файл в каркасе	Задача	Раздел
Главный цикл	`orchestrator.py`: `_drive`	по статусу выбирать следующий шаг	6
Шаг анализа	`orchestrator.py`: `_analyze`; `sanitize.py`	контекст → assess → проверка → решение	7
Шаг выполнения	`orchestrator.py`: `_execute_step`; `prompt.py`	промпт → LLM → инструменты	8
Инструменты	`tools/registry.py`, `tools/builtin.py`	реестр, проверка аргументов, выполнение	8
Сохранение и восстановление	`orchestrator.py`: `_save`, `resume`, `_repair_transcript`, `cancel`	запись состояния, продолжение после сбоя, отмена	9
Лимиты и ошибки	`config.py`, `_call_llm`, `_fail`	лимиты, повторы вызова LLM, завершение с ошибкой	10

Файлы указаны по каркасу из архива `harness_orchestrator.zip`. В нём все эти части уже написаны и покрыты тестами — можно читать код параллельно с этим документом.

5. Статусы задачи и конечный автомат

Зачем статусы

У каждой задачи в базе есть поле `status`. Оно отвечает на вопрос «что с задачей происходит сейчас и что с ней делать дальше». Статус нужен по трём причинам:

1. **Чтобы продолжить задачу после перерыва.** Оркестратор читает задачу из базы, смотрит на статус и сразу знает, какой шаг выполнять. Не нужно ничего помнить в памяти процесса.
2. **Чтобы понять, что значит новое сообщение.** Если в чате есть задача в статусе `waiting_user`, сообщение — ответ на вопрос. Если задача в `executing`, новое сообщение принять нельзя.

3. **Чтобы ловить ошибки в коде.** Если код по ошибке попытается перевести задачу из `new` сразу в `completed`, проверка перехода сразу бросит исключение, и баг будет заметен в тестах.

Семь статусов

Статус	Что означает	Что делает оркестратор, увидев его
`new`	задача только что создана	переводит в `analyzing`
`analyzing`	нужно проверить, хватает ли данных	выполняет шаг анализа (раздел 7)
`waiting_user`	заданы вопросы, ждём ответа	останавливается; продолжит, когда придёт ответ
`executing`	данных хватает, идёт работа с LLM и инструментами	выполняет один шаг выполнения (раздел 8)
`completed`	задача выполнена, ответ отправлен	ничего, задача закрыта
`failed`	задачу не удалось выполнить; причина в поле `failure`	ничего, задача закрыта
`cancelled`	задачу отменили	ничего, задача закрыта

Последние три статуса называются **терминальными**: из них задача уже никуда не переходит. Статуса `cancelled` нет в исходном MVP-документе. Я добавил его, чтобы отмена пользователем не выглядела как ошибка системы.

Разрешённые переходы

Конечный автомат — это просто таблица «из какого статуса в какие можно перейти» и функция, которая проверяет каждый переход по этой таблице.

```
ALLOWED = {
    NEW:          {ANALYZING, FAILED, CANCELLED},
    ANALYZING:    {WAITING_USER, EXECUTING, FAILED, CANCELLED},
    WAITING_USER: {ANALYZING, FAILED, CANCELLED},
    EXECUTING:    {ANALYZING, COMPLETED, FAILED, CANCELLED},
    COMPLETED:    set(),
    FAILED:       set(),
    CANCELLED:    set(),
}

def transition(task, to, reason):
    if to not in ALLOWED[task.status]:
        raise InvalidTransition(f"{task.status} -> {to}")
    new_task = копия task со status=to
    return new_task, Transition(from_status=task.status, to_status=to,
reason=reason)
```

Функция не меняет переданную задачу и ничего не пишет в базу. Она возвращает новую копию и описание перехода (откуда, куда, почему), а сохранением занимается вызывающий код. Так сделано, чтобы смена статуса и сопутствующие данные (например, вопросы) сохранялись одной транзакцией.

Каждый переход и его причина

Переход	Когда происходит
`new → analyzing`	сразу после создания задачи
`analyzing → waiting_user`	Clarification Engine решил спросить (ASK)
`analyzing → executing`	Clarification Engine решил, что данных хватает (PROCEED)
`analyzing → failed`	Clarification Engine отказался (ABORT), либо исчерпан лимит вопросов, либо ошибка в модуле
`waiting_user → analyzing`	пришёл ответ пользователя (даже непонятный — снова проверяем данные)
`waiting_user → cancelled`	пользователь написал «отмена» или начал новую задачу
`executing → completed`	LLM вернула ответ без запросов инструментов

Переход	Когда происходит
`executing → analyzing`	LLM во время работы обнаружила, что чего-то не хватает, и вызвала `request_clarification`
`executing → failed`	превышен лимит или LLM недоступна
`любой нетерминальный → cancelled`	вызван `cancel(task_id)`

Схема этих переходов — в разделе 6 вкладки «Оркестрация».

История переходов

Вместе с каждым сохранением задачи в отдельную таблицу `task_snapshots` пишется снимок: номер версии, статус, параметры, счётчики и переход с причиной. Снимки никогда не меняются. Из них видна вся история задачи, например:

```
v1: new -> analyzing (start)
v2: analyzing -> waiting_user (ask round 1: 1 question)
v3: waiting_user -> analyzing (answers received 1/1)
v4: analyzing -> executing (enough data)
v11: executing -> completed (final answer)
```

Текущее состояние всегда берётся из таблицы `tasks`, а не из снимков. Снимки нужны только для отладки и анализа.

6. Приём сообщения и главный цикл

Шаг 1. Приём сообщения: `submit_message`

Любое сообщение пользователя попадает в оркестратор через этот метод. Сначала нужно понять, что это за сообщение. Для этого делаются две проверки:

```
async def submit_message(self, msg):
    async with storage.unit_of_work() as uow:          # одна транзакция на
чтение
        # 1) не повтор ли это уже полученного сообщения?
        if msg.client_message_id:
            task_id = await uow.inbox.get(msg.client_message_id)
            if task_id:
                return RunResult(task=await uow.tasks.get(task_id),
duplicate=True)
        # 2) есть ли в чате незаконченная задача?
        active = await uow.tasks.find_active_by_chat(msg.chat_id)

    if active is None:
        return await self._start_new(msg)                # новая задача
    if active.status == WAITING_USER:
        return await self._handle_answer(active.id, msg)  # ответ на вопрос
    raise TaskBusy(active.id)                            # задача
выполняется, сообщение не принимаем
```

Проверка на повтор. Интерфейс может отправить одно и то же сообщение дважды: пользователь дважды нажал кнопку или первый запрос дошёл, а ответ на него потерялся, и интерфейс повторил запрос. Без проверки создалась бы вторая задача или ответ засчитался бы дважды. Поэтому у каждого сообщения есть уникальный `client_message_id`, а в таблице `inbox` хранится, какие сообщения уже обработаны и в какую задачу они попали. Свойство «повторный вызов ничего не меняет» называется **идемпотентностью**.

Одна активная задача на чат. В MVP в одном чате одновременно может быть только одна незаконченная задача. Тогда смысл сообщения определяется однозначно: если задача ждёт ответа — это ответ, если задачи нет — новая задача. Если задача прямо сейчас выполняется, сообщение отклоняется ошибкой `TaskBusy`. Интерфейс показывает «подождите, идёт работа».

Шаг 2а. Новая задача: `_start_new`

```
async def _start_new(self, msg):
    task = Task(chat_id=msg.chat_id, user_id=msg.user_id,
                project_id=msg.project_id, request_text=msg.text)   # status
= NEW
    async with storage.unit_of_work() as uow:
        await uow.tasks.create(task)
        await uow.messages.add(Message(task_id=task.id, role=USER,
kind=REQUEST, content=msg.text))
        if msg.client_message_id:
            await uow.inbox.put(msg.client_message_id, task.id)
        await uow.snapshots.add(TaskSnapshot.of(task, None))
    events = []
    task = await self._drive(task.id, events, trigger=INITIAL)
    return RunResult(task=task, events=events)
```

Задача, сообщение и запись в `inbox` сохраняются в одной транзакции. Если процесс упадёт посередине, не останется задачи без сообщения или сообщения, помеченного как обработанное, без задачи. После этого запускается главный цикл `_drive`.

Шаг 2б. Ответ на вопрос: `_handle_answer`

1. Из базы читаются задача, её открытые вопросы (со статусом `pending`) и проекты чата.
2. Вызывается `clarifier.parse_answers(task, pending, message, projects)`. Он возвращает **намерение** (intent) и разобранные ответы.
3. Дальше действие зависит от намерения:

Намерение	Пример сообщения	Что делает оркестратор
`ANSWER`	«backend»	сохраняет ответы, записывает параметры в задачу с пометкой «от пользователя», вопросы с ответом → `answered`, без ответа → `skipped`, статус → `analyzing`, запускает `_drive`

Намерение	Пример сообщения	Что делает оркестратор
`UNCLEAR`	«не знаю»	то же, что и для `ANSWER`, но ответов нет. Анализ запускается снова, и Clarification Engine решит, спросить ли ещё раз. Раунды ограничены, поэтому бесконечного цикла не будет
`CANCEL`	«отмена»	статус → `cancelled`, все открытые вопросы → `skipped`, событие `task_cancelled`
`NEW_REQUEST`	«лучше покажи последние коммиты»	старая задача → `cancelled`, дальше сообщение обрабатывается как новая задача (`_start_new`)

Результат `parse_answers` оркестратор тоже проверяет. Ответы на вопросы, которых нет среди открытых, отбрасываются. Всем параметрам из ответа принудительно ставится источник «пользователь».

Шаг 3. Главный цикл: `_drive`

`_drive` — сердце оркестратора. Он смотрит на статус задачи, выполняет соответствующий шаг и повторяет это, пока задача не остановится. Остановка — это статус `waiting_user` (ждём человека) или любой терминальный статус.

```
async def _drive(self, task_id, events, trigger, uncertainty=None):
    async with self._locks[task_id]:                    # одну задачу ведёт одна корутина
        deadline = now() + config.max_run_seconds
        task = await self._load(task_id)                # всегда начинаем с чтения из базы
        while not task.status.is_terminal and task.status != WAITING_USER:
            try:
                if now() > deadline:
                    task = await self._fail(task, BUDGET_EXCEEDED, "превышено время", events)
                    break
                if task.status == NEW:
                    task = await self._save_transition(task, ANALYZING, "start")
                elif task.status == ANALYZING:
                    task = await self._analyze(task, events, trigger, uncertainty)
                    trigger, uncertainty = None, None
```

```
                elif task.status == EXECUTING:
                    task, new_uncertainty = await self._execute_step(task,
events)
                    if new_uncertainty:                    # модель попросила
уточнение
                        trigger, uncertainty = EXECUTION_UNCERTAINTY,
new_uncertainty
            except ConcurrencyError:
                task = await self._load(task_id)          # кто-то изменил
задачу (например, отменил)
                break
            except ExternalModuleError as e:
                task = await self._fail(await self._load(task_id),
INTERNAL_ERROR, str(e), events)
        return task
```

Разбор по строкам:

- **`_locks[task_id]`** — блокировка внутри процесса. Она гарантирует, что два одновременных вызова не будут одновременно вести одну задачу.
- **`deadline`** — ограничение на общее время одного запуска цикла (по умолчанию 10 минут). Время ожидания ответа пользователя сюда не входит: когда задача ждёт, цикл уже завершён.
- **Условие `while`.** Цикл идёт, пока задача не закрыта и не ждёт пользователя. Каждая итерация — один шаг, после которого задача уже сохранена в базе.
- **`trigger`** — причина, по которой запускается анализ: `initial` (новая задача), `after_answer` (пришёл ответ) или `execution_uncertainty` (модель во время работы сказала, что чего-то не хватает). Он передаётся в Clarification Engine, чтобы тот понимал ситуацию.
- **`ConcurrencyError`** — кто-то другой изменил задачу, пока цикл делал свой шаг. Чаще всего это отмена. Цикл перечитывает задачу и останавливается, не затирая чужие изменения (подробно — раздел 9).
- **`ExternalModuleError`** — исключение внутри Context Engine или Clarification Engine. Оркестратор не может знать, что там сломалось, поэтому честно завершает задачу с ошибкой и текстом исключения.

Схема цикла со всеми ветками — в начале вкладки «Оркестрация» (третья диаграмма).

7. Шаг анализа (`analyzing`)

Задача шага — решить, можно ли выполнять задачу или сначала нужно что-то спросить. Решение принимает Clarification Engine, а оркестратор готовит для него данные, проверяет ответ и исполняет его.

Предварительно: параметры, пробелы, допущения

У каждой задачи есть поле `frame` («карточка задачи»). В нём накапливается всё, что известно о задаче:

- **`goal`** — цель задачи в формулировке Clarification Engine.
- **`slots`** — параметры задачи. Параметр (в коде — `Slot`) состоит из имени, значения и источника. Пример: `project_id = "p_backend"` , источник — `USER` (сказал пользователь). Источников четыре: `USER` , `TOOL` (узнали через инструмент), `CONTEXT` (нашли в контексте) и `ASSUMPTION` (приняли по умолчанию). Источники упорядочены по надёжности, и более слабый не может перезаписать более сильный. Если пользователь сказал `backend` , найденное в контексте другое значение его не заменит.
- **`open_gaps`** — пробелы, то есть параметры, которые нужны, но неизвестны. Пробел (`Gap`) бывает **блокирующим** (`blocking=True`): без этого параметра задачу выполнять нельзя, как без проекта. Или **неблокирующим**: можно принять разумное значение по умолчанию, как «какие именно тесты».
- **`assumptions`** — допущения: значения, принятые для неблокирующих пробелов, вместе с объяснением. В финальном ответе они обязательно показываются пользователю.

Имена параметров придумывает Clarification Engine. Оркестратор знает только один параметр — `project_id` . Если он заполнен, именно из этого проекта берутся контекст и файлы для инструментов.

Порядок действий в `_analyze`

1. **Сбросить кеш контекста.** После ответа пользователя мог появиться проект, и тогда контекст должен быть другим.
2. **Собрать контекст.** Оркестратор формирует запрос `ContextQuery` :
   - `text` — текст запроса плюс значения параметров от пользователя, например «Проверь тесты проекта p_backend»;
   - `project_id` — действующий проект: из параметра `project_id` , а если его нет — из самой задачи;
   - `scopes` — где искать: `chat` , `project` , `memory` ;

- `token_budget` — максимальный размер контекста, 8 000 токенов.

Затем вызывает `context_engine.gather(query)` и кладёт результат в кеш.

3. **Загрузить из базы** все сообщения задачи, все уже заданные вопросы, все ответы и проекты чата.
4. **Посчитать, сколько ещё можно спрашивать:** `rounds_left = 3 - counters.clarification_rounds`.
5. **Вызвать `clarifier.assess(input)`.** Входные данные: задача с карточкой, контекст, история, заданные вопросы и ответы, проекты, `trigger`, текст неопределённости от модели (если есть), номер раунда, максимум вопросов (3) и `rounds_left`.
6. **Проверить ответ** — см. ниже.
7. **Применить ответ к задаче:** добавить найденные параметры, записать цель и список пробелов, запомнить id использованных блоков контекста, добавить токены, потраченные Clarification Engine, в счётчики задачи.
8. **Исполнить решение** — см. ниже.

Проверка ответа Clarification Engine (`sanitize_assessment`)

Зачем проверять: Clarification Engine наверняка будет использовать LLM, а LLM ошибается. Если движок скажет «выполнять», хотя проект неизвестен, система нарушит своё главное правило. Поэтому оркестратор приводит ответ в соответствие с правилами и пишет в лог каждое исправление, чтобы Родион мог чинить свой модуль.

Сначала оркестратор строит карточку задачи такой, какой она станет после применения найденных параметров. По ней он находит **открытые блокирующие пробелы** — блокирующие пробелы, для которых нет параметра с настоящим источником (USER, TOOL или CONTEXT; допущение не считается). Дальше правила применяются по порядку:

#	Правило	Что делает оркестратор при нарушении	Зачем
1	Допущение нельзя принимать для блокирующего пробела	выбрасывает такое допущение	иначе система «угадает» проект
2	Нельзя спрашивать то, что пользователь уже сказал	выбрасывает такой вопрос	повторный вопрос раздражает пользователя

#	Правило	Что делает оркестратор при нарушении	Зачем
3	Не больше одного вопроса на параметр	оставляет вопрос с наибольшим приоритетом	два вопроса про одно — лишнее
4	Не больше K = 3 вопросов за раз	сортирует по приоритету и обрезает	так требует MVP
5	Если раундов не осталось, спрашивать нельзя	есть открытый блокирующий пробел → отказ (`clarification_exhausted`); нет → выполнять	чтобы уточнение не шло бесконечно
6	Решение «спросить», но после правил 2–4 вопросов не осталось	есть открытый блокирующий пробел → отказ; нет → выполнять	нельзя ждать ответа, не задав вопроса
7	Решение «выполнять», но есть открытый блокирующий пробел	есть вопросы и раунды → спросить; иначе → отказ	главное правило: не додумывать

У всех оставшихся вопросов оркестратор перезаписывает `task_id` и номер раунда на правильные, чтобы ошибка движка в этих полях не попала в базу.

Исполнение решения

ASK — спросить. Счётчик раундов увеличивается на 1, счётчик вопросов — на число вопросов. Из вопросов собирается текст для пользователя: один вопрос — как есть, несколько — нумерованным списком, варианты в скобках. Одной транзакцией сохраняются вопросы, сообщение с текстом вопросов и задача в статусе `waiting_user`. В список событий добавляется `questions_asked`. Главный цикл на этом останавливается.

PROCEED — выполнять. Допущения добавляются в карточку (кроме тех, для которых уже есть настоящий параметр). Задача переводится в `executing` и сохраняется. На следующей итерации главный цикл выполнит первый шаг выполнения.

ABORT — отказаться. Задача переводится в `failed`. Код причины: `clarification_exhausted` (закончились раунды) или `engine_abort` (движок сам решил, что задача невыполнима). Пользователь видит объяснение из `reason`,

например: «Лимит уточнений исчерпан, не определён проект».

8. Шаг выполнения (`executing`) и tool calling

Как работает tool calling

LLM не умеет сама читать файлы или запускать команды. Но в запросе к модели можно передать **список инструментов**: имя, описание и схему аргументов каждого. Тогда модель может вернуть не текст, а **запрос на вызов** (tool call): «хочу вызвать `read_file` с аргументом `path="README.md"`». Модель при этом ничего не выполняет — она только сообщает, что хочет вызвать.

Дальше работает наш код: оркестратор выполняет инструмент и вызывает модель ещё раз. В новом запросе есть всё, что было раньше, плюс два новых сообщения: запрос модели на вызов и результат этого вызова. Модель читает результат и либо просит следующий инструмент, либо отвечает текстом. **Ответ текстом без запросов на вызов означает, что задача решена.**

Пример переписки с моделью во втором ходе нашего сценария:

```
Вызов 1, вход:
  system:    «Ты — агент… не придумывай критичные параметры…»
  user:      «# Задача. Исходный запрос: Проверь тесты проекта.
              Известные параметры: project_id = p_backend (источник: user)
              Допущения: test_scope = all — не сказано, какие тесты
              # Контекст [ctx:ctx_1 | project:file:README.md] …»
  assistant: «Уточняющий вопрос: Какой проект проверить?»
  user:      «Ответ пользователя: backend»
  + список инструментов: list_files, read_file, search_files,
get_project_context,
    search_memory, request_clarification
Вызов 1, выход:
  tool_calls: [{id: "call_7", name: "read_file", arguments: {path:
"README.md"}}]

Оркестратор выполняет read_file → «# backend\nТесты запускаются командой
pytest -q»

Вызов 2, вход: всё то же самое +
  assistant: tool_calls=[call_7: read_file(path="README.md")]
  tool (call_7): «# backend\nТесты запускаются командой pytest -q»
Вызов 2, выход:
  text: «В проекте backend тесты запускаются командой pytest -q …»,
tool_calls: []
```

У каждого запроса на вызов есть `id` (в примере `call_7`). Результат возвращается с тем же `id`, чтобы модель сопоставила его с запросом. API провайдеров требуют, чтобы **на каждый запрос на вызов был ровно один результат**, иначе следующий вызов модели вернёт ошибку. Это важно для восстановления после сбоя (раздел 9).

Как переписка хранится в базе

Вся переписка задачи лежит в таблице `messages`. У каждого сообщения есть тип `kind`:

`kind`	Кто автор	Содержимое	Как попадает в промпт
`request`	пользователь	исходный запрос	внутри карточки задачи
`question`	система	текст уточняющего вопроса	assistant: «Уточняющий вопрос: …»
`answer`	пользователь	ответ на вопрос	user: «Ответ пользователя: …»
`tool_call`	модель	запросы на вызов (`tool_calls`)	assistant с `tool_calls`
`tool_result`	инструмент	результат или «ERROR: …» + `tool_call_id`	tool с `tool_call_id`
`final`	модель	финальный ответ с блоком допущений	не попадает: после него задача закрыта
`error`	система	текст ошибки для пользователя	не попадает

Промпт для модели каждый раз собирается заново из этих сообщений. Поэтому оркестратору не нужно держать переписку в памяти, и после перезапуска модель получит ровно тот же вход.

Сборка промпта (`PromptBuilder.build_execution`)

1. **System** — постоянная инструкция: не придумывать критичные параметры; при новой критичной неопределённости вызвать `request_clarification`; сначала искать ответ в контексте и инструментами; ссылаться на контекст в виде `[ctx:id]`; в конце ответить текстом и перечислить допущения; отвечать на языке пользователя.

2. **Карточка задачи** — исходный запрос, цель, все параметры с источниками и допущения.
3. **Контекст** — блоки от Context Engine с пометкой источника. Карточка и контекст идут одним user-сообщением.
4. **Переписка задачи** — все сообщения из таблицы выше в хронологическом порядке. Два user-сообщения подряд склеиваются в одно: многие API этого требуют.

Порядок — от неизменного к меняющемуся. Начало промпта одинаково на всех шагах задачи, и провайдер может его кешировать — это дешевле и быстрее.

Один шаг выполнения (`_execute_step`) по порядку

1. **Проверить лимиты.** Если сделано уже 12 шагов или потрачено 200 000 токенов, задача завершается с ошибкой `budget_exceeded`.
2. **Получить контекст** из кеша; если кеша нет (например, после перезапуска) — собрать заново.
3. **Прочитать из базы** переписку задачи, действующий проект и список уже выполненных вызовов инструментов.
4. **Собрать промпт и вызвать LLM** со списком инструментов + `request_clarification`. При временных ошибках вызов повторяется (раздел 10). Если так и не получилось — `failed(llm_error)`.
5. **Записать токены** в счётчики задачи.
6. **Разобрать ответ модели.** Возможны три случая:
   - **Нет запросов на вызов** — это финальный ответ. К тексту модели оркестратор сам добавляет блок «Допущения» из карточки задачи. На модель тут не полагаемся: она могла забыть упомянуть допущения. Ответ сохраняется, статус → `completed`, событие `final_answer`.
   - **Среди запросов есть `request_clarification`** — модель во время работы поняла, что ей чего-то не хватает (например, в README нет команды запуска тестов). Это не настоящий инструмент, а сигнал оркестратору. Оркестратор сохраняет запрос модели и сразу закрывает все его вызовы результатами. Для `request_clarification` результат — «запрос передан пользователю», для остальных — «отложено до уточнения», чтобы не сломать требование «на каждый запрос — результат». Статус → `analyzing`. Текст причины из аргумента `reason` уходит в Clarification Engine с `trigger = execution_uncertainty`.
   - **Обычные запросы на вызов** — см. следующий пункт.

7. **Сохранить запрос модели** (`tool_call`-сообщение) до выполнения инструментов.
8. **Для каждого запроса на вызов по порядку:**
   a. Проверить лимит вызовов на задачу (30). Если превышен — инструмент не запускается, результат — ошибка, а после шага задача завершится с `budget_exceeded`.
   b. Проверить повтор. У каждого вызова есть «подпись» — хеш от имени и аргументов. Если такой же вызов уже выполнялся 2 раза, он не запускается, а модель получает ошибку «используй полученный ранее результат». Это защита от зацикливания, когда модель снова и снова читает один и тот же файл.
   c. Записать в базу `ToolRun` со статусом `started` — до запуска. Если процесс упадёт во время выполнения, по этой записи будет видно, что вызов начался, но не закончился.
   d. Выполнить через `ToolExecutor.run` (см. ниже).
   e. Одной транзакцией записать: `ToolRun` со статусом `succeeded` или `failed` и результатом, `tool_result`-сообщение, обновлённые счётчики задачи. Интерфейсу — событие `progress`.
9. **Увеличить счётчик шагов** и сохранить задачу. Статус остаётся `executing`, и главный цикл вызовет следующий шаг.

Ошибка инструмента не завершает задачу. Если файл не найден или аргументы неверные, модель получит текст ошибки как результат и сама решит, что делать: исправить аргументы, попробовать другой файл или ответить тем, что есть. Зациклиться она не сможет: мешают лимиты шагов и повторов.

ToolExecutor: как выполняется один вызов

1. Найти инструмент в реестре по имени. Нет такого → ошибка `unknown tool`.
2. Проверить аргументы по схеме: есть ли все обязательные, нет ли лишних, совпадают ли типы. Ошибки → результат `invalid arguments: missing required argument 'path'` и т.п.
3. Запустить обработчик с ограничением времени (20 с по умолчанию). Не успел → ошибка `timeout`.
4. Любое исключение внутри инструмента перехватывается и превращается в результат с ошибкой. Наружу исключения не выходят.
5. Слишком длинный вывод обрезается до 12 000 символов с пометкой «обрезано N символов», чтобы один файл не занял весь контекст модели.

6. Вернуть `ToolResult(call_id, ok, content, error, duration_ms)`.

Файловые инструменты работают только внутри каталога действующего проекта (`Project.root_path`). Путь вида `../frontend/README.md` или ссылка на файл вне проекта даст ошибку «путь выходит за пределы проекта». Иначе модель могла бы прочитать файлы чужого проекта или системы.

Инструменты MVP

Инструмент	Аргументы	Что возвращает
`list_files`	`path` (по умолчанию `.`), `glob` (маска, например `*.py`), `recursive`	список файлов, по одному на строку
`read_file`	`path` (обязательный), `max_bytes`	текст файла
`search_files`	`pattern` (регулярное выражение), `glob`, `max_results`	строки вида `путь:номер_строки: текст`
`get_project_context`	`query`	заметки и решения проекта через Context Engine
`search_memory`	`query`	записи долговременной памяти и других чатов через Context Engine
`request_clarification`	`reason`, `slot_name`	не выполняется: сигнал оркестратору перейти к уточнению

Все инструменты MVP только читают данные и ничего не меняют (`side_effects = False`). Инструменты, которые что-то меняют (запуск тестов, git), появятся позже с пометкой `side_effects = True`. Эта пометка важна при восстановлении после сбоя (раздел 9).

9. Хранение состояния: транзакции, версия задачи, восстановление

Транзакции

Транзакция — группа изменений в базе, которая применяется целиком или не применяется вообще. В нашем коде она выглядит так:

```
async with storage.unit_of_work() as uow:
    await uow.clarifications.add_questions(questions)
    await uow.messages.add(question_message)
    task = await uow.tasks.update(task_with_status_waiting_user)
# вышли из блока без исключения — все три изменения записаны (COMMIT)
# возникло исключение — ни одно не записано (ROLLBACK)
```

Зачем это нужно. Если бы вопросы сохранились, а статус нет (процесс упал между записями), задача осталась бы в `analyzing` с висящими вопросами. При восстановлении оркестратор снова спросил бы то же самое. Транзакция исключает такие промежуточные состояния.

Правило оркестратора: внутри транзакции никогда не вызываются LLM, инструменты, Context Engine и Clarification Engine. Эти вызовы идут секундами, а транзакция держит блокировки в базе. Поэтому каждый шаг устроен одинаково:

1. короткая транзакция — прочитать нужные данные;
2. без транзакции — долгая работа (LLM, инструмент, assess);
3. короткая транзакция — записать результат и новое состояние задачи.

Версия задачи и защита от одновременных изменений

Между шагами 1 и 3 проходит время, и за это время задачу мог изменить кто-то другой. Типичный случай: пользователь нажал «отменить», пока LLM думала. Если после ответа LLM просто записать задачу, статус `cancelled` затрётся на `executing`, и отмена потеряется.

Решение — **номер версии** в каждой задаче (поле `version`):

- при создании задачи `version = 0`;
- `tasks.update(task)` записывает задачу, только если версия в базе совпадает с `task.version`, и при записи увеличивает версию на 1. В SQL: `UPDATE tasks SET …, version = version + 1 WHERE id = :id AND version = :version`;

- если версия не совпала (обновлено 0 строк), бросается `ConcurrencyError`, и вся транзакция откатывается;
- `update` возвращает новую копию задачи с увеличенной версией. Дальше оркестратор работает с этой копией.

Тот же случай с отменой, по шагам:

1. Цикл прочитал задачу: `status = executing, version = 5`. Вызвал LLM.
2. Пока LLM думает, приходит `cancel(task_id)`. Он читает задачу (`version = 5`), ставит `cancelled` и записывает. В базе теперь `status = cancelled, version = 6`.
3. LLM ответила. Цикл пытается записать свою копию с `version = 5`. В базе уже 6 — `ConcurrencyError`.
4. Цикл ловит ошибку, перечитывает задачу, видит `cancelled` и останавливается. Отмена сохранилась, ответ LLM выброшен.

Так же защищен случай, когда два разных процесса одновременно взялись за одну задачу: проигравший получит `ConcurrencyError` и остановится. Сам `cancel` не берёт блокировку `_locks[task_id]`, иначе ему пришлось бы ждать конца цикла. Если сам `cancel` получит `ConcurrencyError`, он повторит попытку (до 3 раз).

Этот приём называется **оптимистичной блокировкой**, а операция «записать, только если текущее значение равно ожидаемому» — **compare-and-swap (CAS)**. Реализует её Егор в `tasks.update`, а оркестратор правильно обрабатывает `ConcurrencyError`.

Хелпер `_save`

Все сохранения задачи в оркестраторе идут через одну функцию:

```
async def _save(self, uow, task, transition=None):
    saved = await uow.tasks.update(task)                          # CAS по
version
    await uow.snapshots.add(TaskSnapshot.of(saved, transition))  # снимок для
истории
    if transition:
        log.info("transition %s -> %s: %s", transition.from_status,
transition.to_status, transition.reason)
    return saved
```

Благодаря этому каждая запись задачи автоматически проверяет версию, пишет снимок и лог. Забыть какой-то из этих шагов нельзя.

Восстановление после сбоя: `resume`

Процесс может упасть в любой момент. Тогда задача останется в базе в статусе `analyzing` или `executing`, и никто её не ведёт. Метод `resume(task_id)` продолжает такую задачу:

1. Прочитать задачу. Если она закрыта — вернуть как есть. Если ждёт ответа — вернуть событие `questions_asked` с открытыми вопросами, чтобы интерфейс показал их снова.
2. **Починить переписку** (`_repair_transcript`). Если процесс упал после сохранения запроса модели на вызов, но до сохранения результата, в переписке есть запрос без результата. С такой перепиской API модели вернёт ошибку. Поэтому для каждого такого запроса дописывается результат-ошибка:
   - для инструментов без побочных эффектов — «выполнение прервано, при необходимости повтори вызов»: повторное чтение файла ничего не сломает;
   - для инструментов с побочными эффектами — «результат неизвестен, не повторяй без проверки»: команда могла успеть выполниться.

   Записи `ToolRun` со статусом `started` помечаются как `interrupted`.
3. Запустить `_drive` с текущего статуса. Для `analyzing` причина анализа восстанавливается по данным: есть ответы → `after_answer`, нет → `initial`.

Восстановление работает потому, что соблюдены правила из предыдущих разделов: всё состояние лежит в базе, запись идёт после каждого шага, промежуточных состояний нет, и перед выполнением инструмента есть пометка `started`.

Что лежит в базе после нашего сценария

Таблица	Записи
`tasks`	1 задача: `completed`, `project_id = p_backend (USER)`, допущение `test_scope = all`, счётчики токенов и шагов
`messages`	request → question → answer → tool_call → tool_result → final
`clarification_questions`	1 вопрос, `answered`
`clarification_answers`	1 ответ: `p_backend`
`tool_runs`	1 вызов `read_file`, `succeeded`

Таблица	Записи
`task_snapshots`	по одному снимку на каждое сохранение задачи
`inbox`	`m-1 → task`, `m-2 → task`

10. Ошибки, лимиты, асинхронность

Зачем лимиты

LLM не гарантирует, что когда-нибудь закончит. Она может бесконечно запрашивать инструменты, Clarification Engine может бесконечно задавать вопросы, а каждый вызов модели стоит денег и времени. Поэтому у оркестратора есть жёсткие лимиты. Они собраны в классе `OrchestratorConfig`; значения — стартовые, их нужно будет подобрать на настоящих задачах.

Лимит	Значение	Что ограничивает	Что происходит при превышении
`max_questions_per_round`	3	вопросов за один раз	лишние вопросы отбрасываются
`max_clarification_rounds`	3	сколько раз можно спрашивать	спрашивать больше нельзя; если критичное не выяснено — `failed(clarification_exhausted)`
`max_agent_steps`	12	шагов выполнения (вызовов LLM с инструментами)	`failed(budget_exceeded)`
`max_tool_calls`	30	вызовов инструментов на задачу	`failed(budget_exceeded)`
`max_repeated_tool_call`	2	одинаковых вызовов (то же имя + те же аргументы)	вызов не выполняется, модель получает ошибку
`max_tokens_per_task`	200 000	токенов на задачу (включая Clarification Engine)	`failed(budget_exceeded)`
`max_run_seconds`	600	время одного запуска главного цикла	`failed(budget_exceeded)`

Лимит	Значение	Что ограничивает	Что происходит при превышении
`context_token_budget`	8 000	размер контекста от Context Engine	Context Engine отбрасывает наименее полезное
`max_tool_output_chars`	12 000	длину результата одного инструмента	результат обрезается с пометкой
`llm_retries`	3	попыток вызвать LLM при временной ошибке	`failed(llm_error)`
`llm_timeout_s`	60	время ожидания одного ответа LLM	считается временной ошибкой, идёт повтор

Виды ошибок и реакция на них

Основной принцип: каждая ошибка либо исправляется на месте, либо завершает задачу со статусом `failed` и понятной причиной. Задача не должна застревать в непонятном состоянии.

Ошибка	Где возникает	Реакция
Временная ошибка LLM: перегрузка (HTTP 429), сбой сервера (5xx), обрыв связи, таймаут	LLM-адаптер бросает `LLMTransientError`	повтор через 1 с, потом через 2 с; после 3 неудачных попыток — `failed(llm_error)`
Неустранимая ошибка LLM: неверный ключ, слишком длинный запрос	LLM-адаптер бросает `LLMFatalError`	сразу `failed(llm_error)`, повтор не поможет
Неверные аргументы, таймаут или исключение в инструменте	ToolExecutor	результат с ошибкой отдаётся модели, задача продолжается
Ответ Clarification Engine нарушает правила	проверка в оркестраторе	ответ исправляется (раздел 7), в лог пишется предупреждение
Исключение внутри `assess`, `parse_answers` или `gather`	модули Родиона и Егора	`failed(internal_error)` с текстом исключения

Ошибка	Где возникает	Реакция
`ConcurrencyError`	`tasks.update`	цикл перечитывает задачу и останавливается (раздел 9)
Другие ошибки базы: нет соединения и т.п.	Storage	ошибка пробрасывается наружу. Транзакция откатилась, задача в базе осталась в последнем корректном состоянии. Когда база вернётся, задачу можно продолжить через `resume`

Повторные вызовы LLM

```
async def _call_llm(self, messages, tools):
    for attempt in range(config.llm_retries):              # 0, 1, 2
        try:
            return await asyncio.wait_for(
                self.llm.complete(messages, tools,
max_tokens=config.llm_max_tokens),
                timeout=config.llm_timeout_s)
        except (LLMTransientError, asyncio.TimeoutError):
            if attempt < config.llm_retries - 1:
                await asyncio.sleep(config.llm_backoff_s * 2 ** attempt)   #
1 с, потом 2 с
        except LLMFatalError as e:
            raise LLMCallFailed(f"ошибка LLM: {e}")
        except Exception as e:                              # неожиданная
ошибка адаптера
            raise LLMCallFailed(f"ошибка LLM-адаптера: {e}")
    raise LLMCallFailed("LLM недоступна после 3 попыток")
```

Задержка растёт с каждой попыткой. Если провайдер перегружен, частые повторы только усилят нагрузку, а пауза даёт ему время восстановиться. `LLMCallFailed` перехватывается в `_execute_step` и превращается в `failed(llm_error)`.

Завершение с ошибкой: `_fail`

Одна функция для всех случаев статуса `failed`. Одной транзакцией она: переводит задачу в `failed`, записывает код и текст причины в `task.failure`, добавляет сообщение типа `error` («Не удалось выполнить задачу: …») и

помечает неотвеченные вопросы как `skipped`. Затем добавляет событие `task_failed`.

Код причины	Когда
`clarification_exhausted`	закончились раунды уточнения, а критичный параметр неизвестен
`engine_abort`	Clarification Engine сам решил, что задача невыполнима
`budget_exceeded`	превышен лимит шагов, вызовов, токенов или времени
`llm_error`	LLM недоступна
`internal_error`	исключение в Context Engine или Clarification Engine

Асинхронность (`async` / `await`)

Почти всё время оркестратор ждёт: ответа LLM (секунды), базы данных, чтения файла. В обычном (синхронном) коде процесс на это время блокируется и не может обслуживать других пользователей. В асинхронном коде каждое ожидание помечено `await`. Пока одна задача ждёт LLM, тот же процесс выполняет другие задачи и обрабатывает `cancel`.

Что это значит на практике:

- все методы, которые чего-то ждут, объявляются как `async def` и вызываются через `await`;
- методы Storage, Context Engine и Clarification Engine тоже должны быть `async def`. Это требование к Егору и Родиону; Егору для PostgreSQL понадобится асинхронный драйвер (`asyncpg` или SQLAlchemy в async-режиме);
- внутри `async def` нельзя вызывать долгие блокирующие функции (`time.sleep`, синхронные HTTP-запросы): они остановят весь процесс;
- программа запускается через `asyncio.run(main())`.

Из-за асинхронности и нужна блокировка `_locks[task_id]` в `_drive`. Два вызова для одной задачи могут идти одновременно, переключаясь на каждом `await`, а блокировка пускает к задаче только один из них.

11. План реализации моей части

Порядок выбран так, чтобы после каждого этапа было что запустить и проверить, и чтобы Егор и Родион могли начать как можно раньше. Все эти части уже есть в каркасе `harness_orchestrator.zip`: можно взять его за основу и пройти по этапам, разбирая и дорабатывая каждый файл.

1. **Контракты** — `harness/contracts/`. Модели данных (`models.py`), интерфейсы модулей (`ports.py`), исключения (`errors.py`).
   - Готово, когда Егор и Родион прочитали свои вкладки с запросом API, ответили на вопросы и согласились с контрактами. Этот этап идёт первым: без него никто не может начать.
2. **Заглушки** — `harness/fakes/`. Хранение в памяти вместо PostgreSQL, простой Context Engine по словам, Clarification Engine на правилах, LLM с заранее записанными ответами.
   - Готово, когда оркестратор можно запускать без базы, без сети и без кода коллег.
3. **Конечный автомат** — `fsm.py` (раздел 5).
   - Готово, когда тесты проверяют: из терминальных статусов переходов нет, запрещённый переход бросает `InvalidTransition`, исходная задача не меняется.
4. **Приём сообщения и главный цикл** — `submit_message`, `_start_new`, `_drive`, `_save`, `_fail` (разделы 6 и 9). Пока без анализа и выполнения: можно временно сразу переводить задачу в `completed`.
   - Готово, когда задача создаётся, проходит статусы и сохраняется со снимками; повтор с тем же `client_message_id` не создаёт вторую задачу; сообщение в занятый чат даёт `TaskBusy`.
5. **Шаг анализа** — `_analyze`, `sanitize_assessment`, `_handle_answer` (разделы 6 и 7).
   - Готово, когда с двумя проектами задача останавливается в `waiting_user` с вопросом про проект; после ответа идёт дальше; «отмена» отменяет; после трёх непонятных ответов задача завершается с `clarification_exhausted`, а LLM ни разу не вызвана; тесты на каждое правило проверки из раздела 7 проходят.
6. **Инструменты** — `tools/registry.py`, `tools/builtin.py` (раздел 8).
   - Готово, когда неверные аргументы, неизвестный инструмент, таймаут и путь вне проекта возвращают ошибку-результат, а не исключение.
7. **Шаг выполнения** — `_execute_step`, `_complete`, `_call_llm`, `prompt.py` (разделы 8 и 10).

- Готово, когда сценарий «Проверь тесты проекта» доходит до `completed` с блоком допущений; `request_clarification` возвращает задачу в анализ; третий одинаковый вызов не выполняется; превышение шагов даёт `budget_exceeded`; временные ошибки LLM повторяются, неустранимые сразу дают `llm_error`.

8. **Отмена и восстановление** — `cancel`, `resume`, `_repair_transcript` (раздел 9).
   - Готово, когда отмена во время вызова LLM останавливает цикл и инструменты после неё не запускаются; задача, «упавшая» посреди вызова инструмента, доводится до конца через `resume`.
9. **Настоящий LLM-адаптер** — класс с методом `complete` для выбранного провайдера. Превращает наши `LLMMessage` и `ToolSpec` в формат API, ответ — в `LLMResponse`, а ошибки API раскладывает на `LLMTransientError` и `LLMFatalError`.
   - Готово, когда демо-сценарий проходит с настоящей моделью.
10. **Интерфейс** — простая консольная программа: читает строку, вызывает `submit_message`, печатает события.
11. **Интеграция** — заменить заглушки на модули Егора и Родиона по мере готовности.
    - Готово, когда все тесты оркестратора проходят с настоящим Storage и в логах нет предупреждений `contract_violation`.

Чтобы коллеги могли проверить себя до интеграции, в каркасе есть готовые наборы проверок: `harness/testing/storage_contract.py` для Егора и `harness/testing/clarifier_contract.py` для Родиона. Их можно отдать сразу вместе с запросами API.

12. Словарь терминов

Термин	Что означает
Задача (`Task`)	одна просьба пользователя со всем её состоянием: статус, параметры, счётчики, версия
Карточка задачи (`TaskFrame`)	цель, параметры, пробелы и допущения задачи
Параметр / слот (`Slot`)	известное значение с источником, например `project_id = p_backend` от пользователя

Термин	Что означает
Пробел (`Gap`)	нужный, но неизвестный параметр
Блокирующий пробел	пробел, без которого задачу выполнять нельзя; по нему нужно спрашивать, допущение запрещено
Допущение (`Assumption`)	значение по умолчанию для неблокирующего пробела; всегда показывается в финальном ответе
Раунд уточнения	один раз, когда система задаёт пользователю 1–3 вопроса и ждёт ответа
`assess`	метод Clarification Engine: хватает ли данных? Ответ — ASK, PROCEED или ABORT
`parse_answers`	метод Clarification Engine: что означает сообщение пользователя — ответ, отмена, новый запрос или непонятно
`trigger`	причина анализа: `initial` , `after_answer` , `execution_uncertainty`
Блок контекста (`ContextBlock`)	кусок сведений из чата, проекта или памяти с указанием источника и оценкой полезности
Токен	единица текста для LLM, примерно 3–4 символа; ими измеряются размер запроса и стоимость
Промпт	список сообщений, который отправляется в LLM за один вызов
Tool calling	механизм, при котором LLM возвращает запрос на вызов инструмента, а выполняет его наш код
`ToolCall` / `ToolResult`	запрос модели на вызов (имя + аргументы + id) и результат с тем же id
`ToolRun`	запись в базе об одном выполнении инструмента: `started` → `succeeded` / `failed` / `interrupted`
`request_clarification`	псевдоинструмент, через который LLM сообщает оркестратору, что ей не хватает данных

Термин	Что означает
Побочный эффект (`side_effects`)	инструмент что-то меняет (запускает команду, пишет файл); после сбоя такой вызов нельзя вслепую повторять
Транзакция / unit of work	группа изменений в базе, которая применяется целиком или не применяется вообще
COMMIT / ROLLBACK	применить / отменить все изменения транзакции
Версия задачи (`version`)	счётчик, который растёт при каждом сохранении; по нему видно, что задачу уже изменил кто-то другой
CAS (compare-and-swap)	«записать, только если версия в базе равна ожидаемой»
`ConcurrencyError`	ошибка CAS: версия не совпала, задачу изменили раньше
Снимок (`TaskSnapshot`)	неизменяемая копия состояния задачи при каждом сохранении; нужна для отладки
Идемпотентность	повторная отправка того же сообщения ничего не меняет; обеспечивается `client_message_id` и таблицей `inbox`
Терминальный статус	`completed` , `failed` , `cancelled` — из них переходов нет
Контракт	общие модели данных и список методов модулей, о которых договорилась команда
Заглушка (fake)	простая реализация модуля с теми же методами для тестов и разработки до того, как готов настоящий